

PRÁCTICA 3

Hecho por: Enrique Gómez Fernández y Daniel Birsan, P10

1-D. Cuestiones sobre QuickSelect y QuickSort:

1. Argumentar que MergeSort ordena una tabla de 5 elementos en a lo sumo 8 comparaciones de clave.

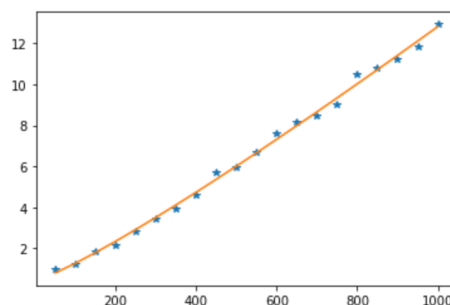
Esto se debe a que el caso peor de mergesort hace todas las comparaciones posibles, esto es $N-1$ comparaciones para cada tabla, y como las tablas se van dividiendo a la mitad, El resultado será $4 + 1 + 2 + 1 = 8$. También podemos seguir la ecuación: $W_{ms}(N) \leq W_{ms}(N/2) + W_{ms}(N/2) + N-1$, y si lo hacemos para $N = 5$ el resultado es 8.

2. En realidad, en `qsel_5` solo queremos encontrar la mediana de una tabla de 5 elementos, pero no ordenarla. ¿Podríamos reducir así el número de comparaciones de clave necesarias?

Pongamonos en el caso peor, todas las permutaciones de 5 elementos están ordenadas y se realizan el número máximo de cdc's, si no queremos ordenar la tabla sino que solo queremos separarla en tablas de 5 elementos las comparaciones de clave se reducirían 5 cdc's por mediana 5, entonces, en términos genéricos si cada mediana reduce 5 cdc's, las cdc's totales serán $0.2 \cdot N$ respecto a las cdc's realizadas al ordenar las medianas y por tanto, la tabla.

3. ¿Qué tipo de crecimiento cabría esperar en el caso peor para los tiempos de ejecución de nuestra función `qsort_5`? Intenta justificar tu respuesta primero experimental y luego analíticamente.

Para quick sort tenemos que el caso peor se da cuando la permutación está ordenada, esto es $N-1$ cdc's, por tanto, una representación prácticamente lineal debería ser el resultado analítico de W_{qs} :



II-C. Cuestiones sobre las funciones de programación dinámica:

1. El problema de encontrar la máxima subsecuencia común (no consecutiva) se confunde a veces con el de encontrar la máxima subcadena común consecutiva. Véase, por ejemplo, la entrada Longest common substring problem en Wikipedia. Describir un algoritmo de programación dinámica para encontrar esta subcadena común máxima consecutiva entre dos cadenas S y T, y aplicarlo “a mano” para encontrar la subcadena consecutiva común más larga entre las cadenas bahamas y bananas.

En esta práctica hemos implementado la máxima subsecuencia común no consecutiva entre dos strings, con esta implementación cambiaremos la comparación entre las letras y añadiremos un flag para que si la siguiente letra a la letra igual es distinta entre los strings no la almacenaremos en el buffer, mientras el flag esté a 1, almacenaremos las letras en el nuevo string, si está a cero y la longitud de la cadena del nuevo string es mayor que 1, no almacenaremos más letras, por el contrario, si es igual a 1, cambiaremos las letras necesarias hasta que el flag esté a 1 o se acabe el bucle por haber recorrido todo el primer string.

2. Sabemos que encontrar el mínimo número de multiplicaciones numéricas necesarias para multiplicar una lista de N matrices tiene un coste $O(N^3)$ pero ahora queremos estimar dicho número $v(N)$ de manera más precisa. Estimar en detalle suficiente dicho número mediante una expresión $v(N) = f(N) + O(g(N))$, con f y g funciones tales que $|v(N) - f(N)| = O(g(N))$.